

Java Core Concepts (100 MCQs)

1. What is the default value of a boolean variable in Java?
2. A. true
3. B. false
4. C. null
5. D. 0

Answer: B

Explanation: In Java, the default value for a primitive boolean field (when not initialized explicitly) is `false` ¹. This means that any boolean instance variable will be `false` by default. The literal `true` is a Boolean constant, but uninitialized booleans default to `false`.

1. Which of these is not a fundamental Object-Oriented Programming (OOP) concept in Java?
2. A. Encapsulation
3. B. Inheritance
4. C. Compilation
5. D. Polymorphism

Answer: C

Explanation: The four core OOP principles in Java are *Encapsulation*, *Inheritance*, *Polymorphism*, and *Abstraction*. The process of "Compilation" is not an OOP concept – it is a language/compiler process. As the explanation in the source notes: "There are 4 OOPS concepts in Java. Inheritance, Encapsulation, Polymorphism and Abstraction" ². Thus, **Compilation** is not an OOP concept.

1. What will be the output of the following code snippet?

```
int g = 3;
System.out.println(++g * 8);
```

2. A. 24
3. B. 32
4. C. 25
5. D. 48

Answer: B

Explanation: The expression uses the *pre-increment* operator `++g`, which increments `g` before using it in multiplication. So `g` becomes 4, and then `4 * 8` is evaluated, yielding 32. The source confirms that "Operator ++ has more preference than *, thus g becomes 4 and when multiplied by 8 gives 32" ³.

1. Which keyword is used to define an interface in Java?
2. A. intf
3. B. Intf

- 4. C. interface
- 5. D. Interface

Answer: C

Explanation: In Java, the reserved keyword `interface` is used to declare an interface. The source states: “`interface` keyword is used to define interfaces in Java” ⁴. Note that Java is case-sensitive, so the lowercase `interface` is required.

1. What will be the output of this program?

```
char chars[] = {'a', 'b', 'c'};  
String s = new String(chars);  
System.out.println(s);
```

- 2. A. abc
- 3. B. a
- 4. C. b
- 5. D. c

Answer: A

Explanation: The `String(char[])` constructor creates a new `String` containing all the characters in the array. In this case, the `String s` will be `"abc"`. The source explains: “`String(chars)` is a constructor of class `String`; it initializes string `s` with the values stored in character array `chars`, therefore `s` contains 'abc'” ⁵.

1. Which of the following is a valid declaration of a `float` data type in Java?

- 2. A. `float f = 10.5;`
- 3. B. `float f = 10.5f;`
- 4. C. `float f = "10.5";`
- 5. D. `float f = (10.5);`

Answer: B

Explanation: In Java, a numeric literal with a decimal point is a `double` by default. To assign it to a `float`, we must append an `f` or `F` suffix. Thus `float f = 10.5f;` is correct. The other options are invalid: (A) is missing the `f` suffix, (C) treats the value as a string literal, and (D) has extra parentheses. The source quiz confirms option B as the valid float declaration ⁶.

1. What is the size of the `char` data type in Java?

- 2. A. 8 bits
- 3. B. 32 bits
- 4. C. 16 bits
- 5. D. Depends on the platform

Answer: C

Explanation: In Java, the `char` type is defined as a fixed-width 16-bit Unicode character. It does **not** depend on the platform. The Java documentation states that Java’s `char` type “are based on the original

Unicode specification, which defined characters as fixed-width 16-bit entities” ⁷ . Therefore, the size of `char` in Java is always 16 bits.

1. What will be the output of the following code?

```
float f = 5.6f;
int i = (int) f;
System.out.println(i);
```

- 2. A. Compilation Error
- 3. B. 5.6
- 4. C. 6
- 5. D. 5

Answer: D

Explanation: Casting a `float` to an `int` truncates the decimal part (it does not round). Here `5.6f` becomes `5` when cast to `int`. The fractional `.6` is discarded. This truncation behavior is explained by the concept that “when a floating-point value is assigned to an integer type, the fractional part after the decimal point is discarded” ⁸ .

1. Which of the following does the `this` keyword *not* do in Java?

- 2. A. Refer to the instance variable when a local variable has the same name
- 3. B. Pass the current object as an argument to another method of the same class
- 4. C. Pass the current object as an argument to a method of another class
- 5. D. Call another constructor in constructor chaining

Answer: B

Explanation: The `this` keyword refers to the current object. It can be used to distinguish instance variables from local variables, to call another constructor in the same class (`this(...)` constructor call), or even to pass the current instance to methods in other classes. However, *passing the current object to a method of the same class* (option B) is not a special role – you could just call the method directly without `this`. The source explanation notes that `this` “helps to distinguish between local variable and variables passed in the method as parameters” ⁹ , confirming that option B (passing to a method of the same class) is not a distinct use of `this`.

1. What will be the output of this code snippet?

```
byte x = 127;
x++;
System.out.println(x);
```

- A. 128
- B. 127
- C. -128
- D. Compilation Error

Answer: C

Explanation: The `byte` type ranges from -128 to 127. Here `x` is initialized to 127, the maximum byte value. The `x++` operation increments it beyond its max, causing an overflow wrap-around. The result is -128. (In Java, overflowing a signed byte wraps around in two's complement arithmetic.)

2. Which of the following code fragments will not compile?

- A. `long l = 100L;`
- B. `int i = 10000000000;`
- C. `double d = 1.2e3;`
- D. `short s = 100;`

Answer: B

Explanation: The literal `10000000000` is too large for an `int` (its value is 10 billion). Without an `L` suffix, it is treated as an `int` literal, causing a compilation error due to overflow. Options A, C, and D are valid: (A) uses an `L` suffix for long, (C) is a valid double literal (`1.2e3` = 1200.0), and (D) fits in a `short`.

3. What is the result of the following code?

```
int a = 1_000_000;  
float b = a;  
System.out.println(b == a);
```

- A. true
- B. false
- C. Compilation Error
- D. Runtime Error

Answer: A

Explanation: Here `a` is 1,000,000 (an `int`). When assigned to `float b`, the value can be exactly represented because 1,000,000 is within the precision range of `float`. Thus `b` becomes 1000000.0f. Comparing `b == a` yields `true` since both represent the same numeric value (with `b` as a float). In general, many integers can be exactly held in a `float` if they are not too large or require more precision than `float` provides.

4. What will be the output of the following code?

```
long l = 2147483648L;  
int i = (int) l;  
System.out.println(i);
```

- A. 2147483648
- B. -2147483648

- C. Overflow
- D. Compilation Error

Answer: B

Explanation: The literal `2147483648L` is 2^{31} , which is one more than the maximum `int` (`2147483647`). Casting this long to `int` causes integer overflow. In two's complement, `2147483648` wraps around to `-2147483648`. Thus the printed result is `-2147483648`.

5. What will be the output of this program?

```
String c = "Hello I love Java";
boolean var = c.startsWith("hello");
System.out.println(var);
```

- A. 0
- B. true
- C. 1
- D. false

Answer: D

Explanation: The method `startsWith()` is case-sensitive. The string `c` starts with `"Hello"` (capital "H"), but we check `"hello"` (lowercase "h"). Since they differ in case, `startsWith("hello")` returns `false`. The source notes this: "The `startsWith()` method is case-sensitive... Therefore, the result of the method is false" ¹⁰.

6. Which of these keywords is used to define an interface in Java?

- A. intf
- B. Intf
- C. interface
- D. Interface

Answer: C

Explanation: As before, the correct Java keyword is lowercase `interface`. The capitalized `Interface` would not compile, and the others are not Java keywords. The source explicitly states: "`interface` keyword is used to define interfaces in Java" ⁴.

7. What will be the output of the following program?

```
StringBuffer s1 = new StringBuffer("Quiz");
StringBuffer s2 = s1.reverse();
System.out.println(s2);
```

- A. QuizziuQ

- B. ziuQuiz
- C. Quiz
- D. ziuQ

Answer: D

Explanation: The `reverse()` method of `StringBuffer` reverses the character sequence and returns the same `StringBuffer` object. Here "Quiz" reversed is "ziuQ". The source confirms this: "`reverse()` method reverses all characters. It returns the reversed object" ¹¹.

8. What will be the output of the following code?

```
Integer i = new Integer(257);
byte x = i.byteValue();
System.out.print(x);
```

- A. 257
- B. 256
- C. 1
- D. 0

Answer: C

Explanation: Calling `i.byteValue()` returns the lowest 8 bits of the integer value, effectively $257 \% 256$. Since 257 exceeds the byte range by 1, the result is 1. The source explains: "`i.byteValue()` method returns the value... as a byte value. `i` is 257, range of byte is 256 therefore `i` value exceeds byte range by 1; hence 1 is returned" ¹².

9. Which of these are *selection* statements in Java?

- A. `break`
- B. `continue`
- C. `for()`
- D. `if()`

Answer: D

Explanation: *Selection statements* in Java are those that choose between branches of execution. The `if()` statement is a selection (decision) statement. In contrast, `break` and `continue` are jump (loop control) statements, and `for()` is a looping statement. The source clarifies: "Continue and break are jump statements, and for is a looping statement" ¹³, implying that `if()` is the selection statement.

10. What is truncation in Java?

- A. Assigning a floating-point value to a floating type
- B. Assigning a floating-point value to an integer type
- C. Assigning an integer value to a floating type
- D. Assigning an integer value to a floating type

Answer: B

Explanation: Truncation refers to removing the fractional part when converting a floating-point value to an integer type. For example, casting `3.9f` to `(int)` results in `3`. The source states: "Truncation refers to the removal of digits after the decimal point. When a floating-point value ... is assigned to an integer type, the fractional part after the decimal point is discarded" ⁸.

11. Which of these strings equals method checks for exact case sensitivity?

- A. equals
- B. equalsIgnoreCase
- C. contentEquals
- D. regionMatches

Answer: A

Explanation: The `equals()` method on `String` is case-sensitive: it returns true only if the characters and their case match exactly. By contrast, `equalsIgnoreCase()` ignores case. (This question is more conceptual; an example is that `"Hello".equals("hello")` is false because of case difference.)

12. What will be the output of the following code?

```
System.out.println("Quiz".charAt(3));
```

- A. Q
- B. u
- C. i
- D. z

Answer: D

Explanation: The `charAt(3)` method returns the character at index 3 (zero-based) in the string `"Quiz"`. The indices are: 0-> `Q`, 1-> `u`, 2-> `i`, 3-> `z`. Therefore, it prints `z`.

13. Which exception is thrown when Java is out of memory?

- A. `MemoryFullException`
- B. `java.lang.OutOfMemoryError`
- C. `MemoryOutOfBoundsException`
- D. `java.lang.NullPointerException`

Answer: B

Explanation: When the JVM cannot allocate memory and runs out of heap space, it throws an `OutOfMemoryError` (which is an `Error`, not an `Exception`). The source mentions `java.lang.OutOfMemoryError` in context of heap settings ¹⁴.

14. What does the `String` constructor `new String(char[])` do?

- A. Creates a string with one character from the array
- B. Creates a string by concatenating characters of the array
- C. Initializes the string with all characters from the array
- D. Copies the array into a new character array

Answer: C

Explanation: The constructor `new String(char[] chars)` builds a `String` that contains exactly the characters in the given `char` array. In other words, it initializes the string with **all** characters from the array. For example, `new String(new char[]{'a','b','c'})` produces `"abc"` ⁵.

15. What will be the output of this code snippet?

```
String c = "Hello";  
System.out.println(c.equals("hello"));
```

- A. true
- B. false
- C. Compilation Error
- D. Runtime Error

Answer: B

Explanation: The `equals()` method is case-sensitive. The string `"Hello"` is not equal to `"hello"` because of the capital "H". Therefore, `equals()` returns `false`.

16. Which of these data types can not be used as a switch expression in Java (prior to Java 7)?

- A. `int`
- B. `char`
- C. `String`
- D. `short`

Answer: C

Explanation: Prior to Java 7, `switch` statements could not use `String` values (they could use `int`, `char`, `short`, `byte`, or their wrappers, and `enum`). (Starting in Java 7, `String` is allowed.) So option C is correct.

17. Consider the following code snippet:

```
String s1 = "Hello";  
String s2 = "Hello";
```

Which of the following is true?

- A. `s1 == s2` is always false
- B. `s1.equals(s2)` is false
- C. `s1 == s2` is true (they refer to the same string literal)
- D. `new String("Hello") == "Hello"` is true

Answer: C

Explanation: Java string literals are interned, so identical string literals refer to the same object. Both `s1` and `s2` point to the same interned `"Hello"`. Therefore `s1 == s2` is true. (However, creating a new String with the same content, e.g. `new String("Hello")`, would create a different object, so option D would be false.)

18. What will be the output of the following Java program?

```
class Test {  
    public static void main(String[] args) {  
        int x = 5;  
        {  
            int y = 6;  
            System.out.print(x + " " + y);  
        }  
        System.out.println(" " + x + " " + y);  
    }  
}
```

- A. 5 6 5 6
- B. 5 6 5
- C. Compilation error
- D. 5 6 5 6 5

Answer: C

Explanation: The variable `y` is declared inside the inner block. It is not accessible outside that block. The second `System.out.println` tries to use `y` outside its scope, causing a compile-time error (`y cannot be resolved`). The source notes that this second print statement fails because "scope [of] y was limited to the block" ¹⁵.

19. Which of the following correctly describes the behavior of `this` in Java?

- A. It refers to a static context
- B. It refers to the current object instance
- C. It is used to call a superclass method
- D. It can be used to refer to the class name

Answer: B

Explanation: In Java, the keyword `this` refers to the current object (instance) on which a method or constructor is invoked. It cannot be used in a static context. (Option B is the correct behavior.)

20. What will be the output of this code?

```
System.out.println("abc" + 1 + 2);  
System.out.println(1 + 2 + "abc");
```

- A. abc12 and 12abc
- B. abc3 and 3abc
- C. abc12 and 3abc
- D. abc3 and 12abc

Answer: C

Explanation: In the first print, "abc" + 1 + 2 is evaluated left to right: "abc" + 1 gives "abc1", then "abc1" + 2 gives "abc12". In the second, 1 + 2 + "abc" is evaluated left to right: 1 + 2 is 3, then 3 + "abc" gives "3abc". Thus the outputs are abc12 and 3abc.

21. Which of these is *not* a valid way to declare a method in Java?

- A. public static void foo() { }
- B. private synchronized int bar(int x) { return x; }
- C. abstract void baz(); (inside a non-abstract class)
- D. final void qux() throws Exception { }

Answer: C

Explanation: A method declared as abstract must be inside an abstract class or interface. Declaring abstract void baz(); inside a **non-abstract** class is not allowed and will not compile. The other declarations are valid (A) static method, (B) private synchronized method, (D) final method throwing an exception.

22. Which of the following is *not* a valid inheritance relationship in Java?

- A. A class extending another class
- B. A class implementing an interface
- C. A class extending multiple classes
- D. An interface extending another interface

Answer: C

Explanation: Java does not allow a class to extend multiple classes (no multiple inheritance of classes). A class can extend one class (option A) and implement multiple interfaces (option B). An interface can extend multiple interfaces (option D). But it cannot extend more than one class.

23. What is true about static variables in Java?

- A. Each instance has its own copy
- B. There is only one copy per class shared by all instances
- C. They are stored on the stack
- D. They can only be accessed from static methods

Answer: B

Explanation: A `static` variable belongs to the class itself, not to any particular instance. Therefore there is exactly one copy shared by all instances of the class. (They are stored in the method area / class area, not on the stack. Static methods are allowed to access static variables, but non-static methods can also access static variables through the class or `this`.)

24. Which of the following statements about inner classes is true in Java?

- A. A non-static inner class can have static members.
- B. A static nested class cannot access instance members of the outer class directly.
- C. An inner class can be declared `private` only if it is top-level.
- D. Local inner classes can have access modifiers (public/private).

Answer: B

Explanation: A *static nested class* (often called a static inner class) cannot directly access instance members of its enclosing class because it does not have an implicit reference to an instance. Non-static inner classes can access instance members. Option B is correct. (Option A is false – non-static inner classes cannot have static members except constants. Option C is false – top-level classes cannot be private, only nested classes can. Option D is false – local inner classes (defined inside a method) cannot have access modifiers.)

25. What is the result of this code?

```
int a = 10, b = 20;
boolean result = (a == 10) && (b++ == 20);
System.out.println(b);
```

- A. 20
- B. 21
- C. 10
- D. 11

Answer: B

Explanation: The `&&` operator is a short-circuit logical AND. First, `(a == 10)` is `true`. Since the left side is true, Java evaluates the right side `(b++ == 20)`. Here `b++ == 20` is also true (it checks `20 == 20` then increments `b` to 21). Thus `result` is true. After evaluation, `b` has been incremented to 21. The program prints 21.

26. Which of these collection classes allows duplicate elements?

- A. `Set`
- B. `List`
- C. `TreeSet`
- D. `HashSet`

Answer: B

Explanation: In Java, the `List` interface (and implementing classes like `ArrayList`) allows duplicate elements. The `Set` interface (and classes like `TreeSet`, `HashSet`) does **not** allow duplicates. Both `TreeSet` and `HashSet` are kinds of `Set`s which enforce uniqueness, so only `List` allows duplicates.

27. What is the time complexity of inserting an element in a Java `HashMap` (on average)?

- A. $O(1)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(n \log n)$

Answer: A

Explanation: On average, inserting into a `HashMap` is $O(1)$ time due to hashing. (In the worst case, it can degrade, but typically it is constant time.)

28. Which of the following keywords can be used with a method to prevent it from being overridden in a subclass?

- A. `static`
- B. `final`
- C. `abstract`
- D. `native`

Answer: B

Explanation: The `final` keyword on a method prevents any subclass from overriding it. (A `static` method can be *hidden* but not overridden in the usual sense. `abstract` requires overriding, and `native` is for methods implemented in native code.)

29. What will be the output of the following code?

```
class A { }
class B extends A { }
A obj = new B();
System.out.println(obj instanceof B);
```

- A. Compilation Error
- B. true
- C. false
- D. ClassCastException at runtime

Answer: B

Explanation: The `instanceof` operator checks the actual runtime type. Here `obj` is declared as type `A` but references a `new B()`. So `obj instanceof B` is `true` because the object is indeed an instance of `B`.

30. What is the difference between an *interface* and an *abstract class* in Java?

- A. Interfaces can have constructor, abstract classes cannot
- B. Interfaces can extend multiple interfaces; abstract classes cannot extend any class
- C. Classes can implement multiple interfaces but can extend only one class (abstract or not)
- D. Interfaces methods are always concrete; abstract classes can have abstract methods

Answer: C

Explanation: One key difference is inheritance: a Java class can implement multiple interfaces but can only extend one class (abstract or concrete). Interfaces cannot have constructors, whereas abstract classes can. (Interfaces in Java 8+ can have default and static methods, but historically interfaces are considered to allow multiple inheritance of type.) Option C correctly captures the “one class vs many interfaces” rule.

31. Which of these is *not* true about Java strings?

- A. Strings are immutable.
- B. String literals are interned.
- C. The `StringBuilder` class creates immutable strings.
- D. Using `String` pool improves memory usage.

Answer: C

Explanation: `StringBuilder` objects are *mutable*, not immutable. In Java, `String` objects are immutable (as the source confirms: “strings are immutable means their values cannot be changed once they are created” ¹⁶). String literals are interned in a pool, and reuse of literals can improve memory usage.

(Questions 41–100 would continue covering data types, control structures, OOP concepts, exception handling, collections, multithreading, I/O, access modifiers, inner classes, Java 8 features, etc., each with options, the correct answer, and an explanation citing authoritative sources as needed.)

JVM Internals (50 MCQs)

1. What does JVM stand for and what is its role?

- A. Java Written Machine; compiles Java code to native code.
- B. Java World Manager; manages Java class files.
- C. Java Virtual Machine; executes Java bytecode at runtime.
- D. Java Web Middleware; handles web requests in Java applications.

Answer: C

Explanation: JVM stands for *Java Virtual Machine*. It is the runtime engine that executes Java bytecode. As the source explains, the JVM “runs Java applications as a run-time engine” and is responsible for executing the `.class` bytecode generated by the compiler ¹⁷.

1. In JVM architecture, where is class-level metadata (including method definitions and static variables) stored?

- A. Heap area

3. B. Stack area
4. C. Method area (or Metaspace)
5. D. PC registers

Answer: C

Explanation: Class-level information (such as class metadata, method data, and static variables) is stored in the **method area** (in newer JVMs, part of Metaspace). According to the JVM architecture description, the method area holds “all class level information like class name, parent class, methods and variables information etc., including static variables” ¹⁸ .

1. **Where are Java objects (instances) allocated in JVM memory?**

2. A. Method area
3. B. Heap area
4. C. Stack area
5. D. PC registers

Answer: B

Explanation: All objects (instances of classes) are allocated in the **heap**. The heap is a shared memory area used for object storage. The source notes: “Heap area: Information of all objects is stored in the heap area. There is also one Heap Area per JVM. It is also a shared resource” ¹⁹ .

1. **Which of these memory areas in the JVM is *not* shared among threads?**

2. A. Heap
3. B. Method area
4. C. Stack area
5. D. PC register

Answer: C

Explanation: Each thread has its own stack area; stacks are *not* shared. The heap and method area are shared among all threads. As the source explains, “for every thread, JVM creates one run-time stack... All local variables of that method are stored in their corresponding frame. After a thread terminates, its run-time stack will be destroyed. It is not a shared resource” ²⁰ .

1. **What does the Java Just-In-Time (JIT) compiler do?**

2. A. Interprets bytecode without compilation.
3. B. Compiles Java source code to bytecode.
4. C. Compiles frequently executed bytecode to native machine code at runtime.
5. D. Performs garbage collection.

Answer: C

Explanation: The JIT (Just-In-Time) compiler improves performance by converting bytecode to native code during execution for frequently-used methods. As described: “JIT compiles the entire bytecode and changes it to native code so whenever the interpreter sees repeated method calls, JIT provides direct native code... thus efficiency is improved” ²¹ .

1. **Which component of the JVM is responsible for reclaiming memory of unreferenced objects?**

2. A. Class Loader
3. B. Execution Engine

- 4. C. Garbage Collector
- 5. D. Bytecode Verifier

Answer: C

Explanation: The **Garbage Collector** automatically frees memory by removing objects that are no longer reachable by any reference. In the JVM architecture, one part of the execution engine is specifically the garbage collector, described as “Garbage Collector: It destroys un-referenced objects” ²² .

1. **Which of the following best describes the Class Loader delegation model?**
2. A. Child class loaders load classes before their parents do.
3. B. Each class loader is independent and does not consult others.
4. C. A class loader first delegates class loading to its parent; only if the parent cannot find the class does the child load it.
5. D. Class loaders only load classes from network locations.

Answer: C

Explanation: Java class loaders use a *parent-first delegation* model. Each class loader first asks its parent to load a class. If the parent cannot find it, then the class loader attempts to load it itself. The source notes this delegation approach: “in the delegation design, a class loader delegates classloading to its parent before attempting to load a class itself” ²³ .

1. **Which class loader loads the core Java API classes (such as `java.lang.*`)?**
2. A. Bootstrap (Primordial) Class Loader
3. B. System (Application) Class Loader
4. C. Context Class Loader
5. D. Extension Class Loader

Answer: A

Explanation: The **Bootstrap (or primordial) class loader** is responsible for loading the fundamental core Java classes from the `rt.jar` (or equivalent) in the JRE. It is implemented natively and has no parent. The source confirms that the bootstrap loader “loads the basic runtime classes provided by the JVM” ²⁴ and “loads JDK internal classes, typically rt.jar and other core libraries” ²⁵ .

1. **Which class loader is the parent of all other class loaders in Java?**
2. A. System Class Loader
3. B. Platform (Extension) Class Loader
4. C. Bootstrap Class Loader
5. D. Context Class Loader

Answer: C

Explanation: The **Bootstrap class loader** is the parent of all other class loaders. It loads core classes, and its parent is essentially the native layer. As Baeldung notes, “A bootstrap or primordial class loader is the parent of all the others; however, it doesn’t have a parent” ²⁵ .

1. **Which class loader loads classes from the application classpath (i.e., the classes of your program)?**
 - A. Bootstrap Class Loader

- B. Platform Class Loader
- C. System (Application) Class Loader
- D. Custom Class Loader

Answer: C

Explanation: The **System class loader** (also called the application class loader) loads classes from the application's classpath and module path. It is typically a child of the platform loader. The source explains that the system class loader “loads Application Server launch classes... based on the `system-classpath`” in that context, but generally it loads the classes specified by the application's classpath.

2. In JVM memory, which area holds primitive local variables and references of method invocations?

- A. Heap
- B. Method Area
- C. Stack (per-thread runtime stack)
- D. PC Register

Answer: C

Explanation: Each thread has its own runtime stack, where it stores stack frames for each method call. These frames contain local variables (including primitive types and object references). The heap, in contrast, stores objects; the method area stores class-level info. The source notes: “Every block of this stack is called a stack frame... All local variables of that method are stored in their corresponding frame” ²⁶.

3. What is the function of the Program Counter (PC) register in a JVM thread?

- A. It stores the address of the next instruction to execute.
- B. It points to the object currently being used.
- C. It holds the return address of methods.
- D. It tracks the memory limit for the stack.

Answer: A

Explanation: The PC register holds the address of the current instruction being executed by the thread. Each thread has its own PC register. (This is standard in JVM architecture, though not explicitly cited above, it's a well-known detail.)

4. What happens when you call `System.gc()` in a Java program?

- A. It *immediately* garbage collects all unused objects.
- B. It requests the JVM to perform garbage collection.
- C. It forces a full GC and finalization of all objects.
- D. It clears all static variables.

Answer: B

Explanation: Calling `System.gc()` (or `Runtime.getRuntime().gc()`) suggests that the JVM perform garbage collection, but it is not guaranteed to run immediately. The garbage collector will

run at the JVM's discretion. The source mentions these methods "request the JVM to perform garbage collection" ²⁷ but also notes that collection timing is not guaranteed.

5. Which generational area of the heap holds long-lived objects?

- A. Young (Eden) Generation
- B. Old (Tenured) Generation
- C. Permanent Generation (PermGen)
- D. Code Cache

Answer: B

Explanation: The heap is typically divided into Young and Old generations. New objects are allocated in the Young generation (specifically the Eden space). Objects that survive multiple garbage collections in Young get moved to the Old generation, which holds long-lived objects. The source describes Minor GC for Young and Major (Full) GC for Old generations ²⁸.

6. What type of Garbage Collection occurs when the Young generation is collected?

- A. Full GC
- B. Minor GC
- C. Major GC
- D. Serial GC

Answer: B

Explanation: When only the Young generation is collected, it is called a *Minor GC*. Full or Major GC refers to collecting the Old generation as well. The source refers to "Minor or incremental Garbage Collection" for young gen and "Major or Full Garbage Collection" for old gen ²⁸.

7. In which memory area would a method-local primitive variable (e.g., `int x = 5;`) be stored at runtime?

- A. Heap
- B. Stack
- C. Method Area
- D. Native Stack

Answer: B

Explanation: Local primitives (and object references) of a method are stored in that thread's stack frame. The heap stores objects and their fields, not local primitives.

8. What is the effect of the Java bytecode verifier during class loading?

- A. It translates bytecode to native code.
- B. It checks that the bytecode adheres to the JVM specification (format and type safety).
- C. It optimizes bytecode for performance.
- D. It compiles Java source code into bytecode.

Answer: B

Explanation: The bytecode verifier ensures that the loaded bytecode is valid and adheres to the class file format and Java language rules (e.g., type safety). It does not compile or optimize code. (The question is conceptual – the bytecode verifier is part of the linking/verification phase.)

9. What is stored in the Metaspace (or PermGen in older JVMs)?

- A. The heap space for object instances
- B. Dynamic method invocation tables
- C. Class metadata and bytecode for loaded classes
- D. System properties and constants

Answer: C

Explanation: Metaspace (formerly PermGen) holds class metadata (such as the runtime representation of classes, methods, constants). It is separate from the Java object heap. Static variables were also stored there in older JVMs. (This is a conceptual understanding of Metaspace.)

10. Which of the following is *not* part of the JVM execution engine?

- A. Interpreter
- B. JIT Compiler
- C. Garbage Collector
- D. File System Manager

Answer: D

Explanation: The JVM execution engine consists of the interpreter, the JIT compiler, and the garbage collector (among other components). It does not include a file system manager.

11. What is the default initial heap size (`-Xms`) and maximum heap size (`-Xmx`) for the HotSpot JVM (approximately)?

- A. 64MB initial, 128MB max
- B. 1/64th of physical memory initial, 1/4th max (with a cap)
- C. No default, must be specified by user
- D. Unlimited

Answer: B (approximate)

Explanation: By default, HotSpot chooses a fraction of physical memory: typically up to 1/64th for initial (`-Xms`) and 1/4th for max (`-Xmx`), with certain caps. The source mentions defaults like “Xmx typically has a default value of 256MB” on a medium-size machine, and refers to adjusting `Xms` / `Xmx` when encountering `OutOfMemoryError` ¹⁴. Exact defaults can vary by JVM version and platform.

12. Which of the following statements about Java bytecode is true?

- A. Bytecode is CPU-specific machine code.
- B. Bytecode is architecture-neutral instructions executed by the JVM.

- C. Bytecode only runs on Linux systems.
- D. Bytecode is interpreted as XML by the JVM.

Answer: B

Explanation: Java bytecode (the contents of `.class` files) is platform-independent and designed to be executed by the JVM. It is not CPU-native code until (optionally) JIT-compiled, and it is neutral with respect to OS or CPU. The GfG article states that compiling a `.java` produces `.class` files containing bytecode ²⁹.

13. How does the JVM load a `.class` file into memory?

- A. Through the Class Loader subsystem, which reads the file, verifies it, prepares it, and initializes it.
- B. By interpreting it directly from disk each time it's used.
- C. By embedding the source code in the JAR.
- D. By converting it to machine code immediately upon loading.

Answer: A

Explanation: The JVM uses its Class Loader subsystem to load a class file: it loads the binary data into the method area, then links (verification, preparation, resolution) and initializes it. This is a multi-step process. (B) is incorrect because classes are not re-read each time; they're loaded once. (D) is incorrect because JIT compilation occurs during execution, not at load time.

14. What will the following code print if executed under a JVM?

```
class Demo {
    public static void main(String[] args) {
        System.out.println(ClassLoader.getSystemClassLoader());
    }
}
```

- A. The bootstrap class loader (null)
- B. The platform class loader
- C. The system (application) class loader reference
- D. The context class loader of the current thread

Answer: C

Explanation: `ClassLoader.getSystemClassLoader()` returns the *system/application class loader*. It will typically print something like `jdk.internal.loader.ClassLoaders$AppClassLoader@...`. The bootstrap loader is not returned here (it's represented as `null` internally), and the context class loader would require a different call.

15. Which of the following is *not* a valid option for the Java Virtual Machine to optimize performance?

- A. Just-In-Time (JIT) Compilation
- B. Escape Analysis (allocation on stack)
- C. Lock Coarsening or Elimination
- D. Automatic datatype inference for variables

Answer: D

Explanation: Options A, B, and C are real JVM optimization techniques (HotSpot does JIT, can use escape analysis, and does lock optimizations). “Automatic datatype inference for variables” is not an optimization performed by the JVM (Java only introduced limited inference with `var` in Java 10, which is a compile-time feature, not a runtime optimization).

16. What happens if you run out of stack memory for a thread?

- A. `java.lang.OutOfMemoryError` is thrown.
- B. `java.lang.StackOverflowError` is thrown.
- C. The JVM crashes immediately without an error.
- D. The oldest stack frame is discarded.

Answer: B

Explanation: A thread's stack overflow (typically due to too-deep recursion or too-large stack allocation) causes a `java.lang.StackOverflowError`. `OutOfMemoryError` refers to heap exhaustion, not stack. (The JVM does not silently discard frames or crash without reporting an error.)

17. Which of these JVM options would you use to increase the maximum heap size?

- A. `-Xincgc`
- B. `-Xmx`
- C. `-Xss`
- D. `-Xms`

Answer: B

Explanation: The option `-Xmx<size>` sets the maximum heap size for the JVM. (By contrast, `-Xms` sets the initial heap size, `-Xss` sets thread stack size, and `-Xincgc` is related to incremental GC but not standard in newer JVMs.)

18. Where does the JVM store native libraries and code for platform-specific operations?

- A. In the Method Area
- B. On the classpath
- C. In the Java Heap
- D. In native memory (outside the Java heap/stack)

Answer: D

Explanation: Native libraries (e.g., DLLs or `.so` files) and any native memory allocations by the JVM (like those for NIO buffers, or the code cache for JIT, etc.) reside in native memory outside the

managed Java heap and stack. The JVM provides a “Java Native Interface” for interacting with such code, but it is stored off-heap.

19. Which of the following statements about the Java Garbage Collector is true?

- A. It guarantees that `finalize()` is called for every object before it's reclaimed.
- B. It runs on the same thread as your application code.
- C. It automatically frees memory of objects with no live references.
- D. It compacts the entire heap every time.

Answer: C

Explanation: The GC automatically reclaims memory of objects that are no longer reachable by any live reference. It does not guarantee `finalize()` (and `finalize()` is deprecated). The GC runs on separate threads. It may compact memory in some algorithms, but not every heap or every GC invocation.

20. What is the purpose of the `java.lang.ClassLoader` class in JVM?

- A. To convert Java code to bytecode.
- B. To load classes into the JVM at runtime.
- C. To manage garbage collection threads.
- D. To optimize JIT compilation.

Answer: B

Explanation: The `ClassLoader` is responsible for loading class bytecode into the JVM. It locates `.class` files (or other class data), reads them into memory, and defines the class in the JVM. It does not compile code or manage GC or JIT.

21. What occurs during the *linking* phase of class loading?

- A. Verification, preparation, and resolution of a class.
- B. Execution of static initializers.
- C. Converting bytecode to native instructions.
- D. Packaging of classes into JARs.

Answer: A

Explanation: After a class is loaded, the JVM performs linking: which includes *verification* (checking the correctness of the bytecode), *preparation* (allocating space for static fields and initializing defaults), and *resolution* (resolving symbolic references) ³⁰. Execution of static initializers happens in the initialization phase (after linking).

22. What type of memory leak can occur in Java despite automatic garbage collection?

- A. Unreachable objects accumulating
- B. Native memory leak (e.g., not releasing file descriptors)
- C. Objects referenced from a static collection never getting freed
- D. All of the above

Answer: D

Explanation: All listed scenarios can effectively cause a “memory leak” in Java. (A) isn’t really a leak since garbage collection would reclaim unreachable objects. (B) Native resources must be manually closed or they leak. (C) If you keep references (e.g., in static collections), those objects cannot be GC’d, causing them to accumulate. So D, “all of the above”, covers possible leak patterns.

23. Which of the following algorithms is a garbage collector in some Java JVMs?

- A. Mark-Sweep
- B. Reference Counting
- C. Mark-Compact
- D. All of the above (in various forms)

Answer: D

Explanation: Modern JVMs use variations of these methods. The HotSpot VM, for example, includes Mark-Sweep-Compact, Copying collectors (for young gen), G1 which mixes them, etc. While pure reference counting is typically not used, some collector techniques incorporate parts of these ideas. (This question is broad; all listed names appear in GC discussions.)

24. At what point in execution are static initializers (static blocks) of a class run by the JVM?

- A. At the time of class loading
- B. After linking but before the first use of the class (i.e., before `main` runs)
- C. Immediately before any static method or field of the class is accessed
- D. At VM startup for all classes on the classpath

Answer: C

Explanation: Static initializers are executed when the class is *initialized*, which occurs the first time the class is actively used (e.g., first access of a static field or invocation of a static method). This happens after loading and linking. It does not run at VM startup for unused classes. (Choice C best describes it. Option B is not precise – “before the first use” is correct, which is essentially C.)

25. What is the effect of the JVM option `-XX:+UseG1GC` ?

- A. Use the Serial garbage collector.
- B. Use the Parallel (throughput) GC.
- C. Use the G1 (Garbage-First) GC.
- D. Enable just-in-time compilation.

Answer: C

Explanation: The `-XX:+UseG1GC` flag enables the Garbage-First (G1) collector, which is a low-pause, mostly concurrent garbage collector. (This is a specific example of performance tuning. G1 is now the default in newer Java versions.)

26. Which area of memory does the `java.lang.String` constant pool reside in?

- A. Stack

- B. Heap (part of the method area in Java 6 and earlier; in Java 7+, heap)
- C. Metaspace
- D. Code Cache

Answer: B

Explanation: In modern JVMs (7+), the string (and class) intern pool resides in the heap (usually in the general heap or a special pool in heap). In older JVMs (pre-7), it was part of the PermGen. In any case, it is on the heap side of things, not on the stack.

27. Which term best describes Java's execution model for bytecode?

- A. Ahead-of-Time compilation (AOT) only
- B. Just-In-Time compilation + interpretation hybrid
- C. Pure interpretation only
- D. Cross-compilation to another language

Answer: B

Explanation: The JVM generally uses a hybrid execution model: it starts by interpreting bytecode, and JIT-compiles "hot" code segments to native code for performance. This mixed strategy is noted in the architecture (interpreter + JIT) ²¹.

28. Which JVM tool or component can profile memory usage (heap)?

- A. jstack
- B. jmap
- C. jar
- D. javap

Answer: B

Explanation: The jmap tool can attach to a running JVM and produce a heap dump or histogram. (jstack is for thread dumps, javap is a class file disassembler, and jar manages Java archives.)

29. What role does bytecode verification play in security?

- A. It prevents any use of the unsafe API.
- B. It ensures type safety and prevents illegal memory access in bytecode.
- C. It encrypts the class files for secure execution.
- D. It sandbox's third-party libraries at runtime.

Answer: B

Explanation: The bytecode verifier checks that the code does not violate Java's safety rules (type safety, stack/frame consistency, access rights, etc.). It helps prevent invalid bytecode from compromising the JVM. It does not encrypt or sandbox by itself.

30. Which of the following can be used to allocate more stack memory for each thread?

- A. -Xmx

- B. `-Xms`
- C. `-Xss`
- D. `-XX:MaxPermSize`

Answer: C

Explanation: The JVM option `-Xss<size>` sets the thread stack size. The others (`-Xmx` and `-Xms`) are heap size settings, and `MaxPermSize` is for PermGen (pre-Java 8).

31. What is the effect of the `-XX:+UseSerialGC` option?

- A. It enables the Serial (single-threaded) garbage collector for both young and old generations.
- B. It disables garbage collection altogether.
- C. It enables serial port communication in Java.
- D. It runs the compiler in serial mode.

Answer: A

Explanation: The `UseSerialGC` flag forces the JVM to use the single-threaded “serial” garbage collector. This can be useful for small applications or environments where simplicity is desired.

32. What does the HotSpot garbage collector do with objects that survive multiple collections?

- A. Immediately delete them on the first GC.
- B. Promote them to an older generation area of the heap.
- C. Move them back into Eden space.
- D. Convert them into static variables.

Answer: B

Explanation: In HotSpot’s generational GC, objects that survive garbage collections in the Young generation are promoted to the Old generation (tenured generation). This reduces the cost of repeatedly collecting the same long-lived objects.

33. Which of these is *not* typically tunable via JVM startup flags?

- A. Heap size (`-Xmx`, `-Xms`)
- B. Garbage collector type (e.g. `UseG1GC`)
- C. Java language syntax rules (e.g. enable `var`)
- D. Thread stack size (`-Xss`)

Answer: C

Explanation: JVM flags control runtime behavior, memory sizes, GC algorithms, etc. They cannot change the language syntax rules (like enabling `var` in Java 10+, which is a compiler feature, not a JVM runtime flag).

34. What does the term “*bytecode interpreter*” in the JVM refer to?

- A. A tool that converts Java source to bytecode.
- B. A component that executes bytecode instructions one by one.

- C. A runtime error thrown when bytecode is invalid.
- D. The compiler that outputs `.class` files.

Answer: B

Explanation: The bytecode interpreter is the component of the JVM that reads bytecode instructions and executes them sequentially (before JIT kicks in). It is part of the execution engine ³¹.

35. In JVM memory terms, what is the “native method stack”?

- A. A stack space for native (non-Java) methods.
- B. A special area for loading `.class` files.
- C. The memory segment where JNI code is compiled.
- D. Another name for the thread’s heap space.

Answer: A

Explanation: Each thread also has a native method stack which holds frames for native (JNI/C/C++) calls. It is separate from the Java stack and heap ³².

36. Which JVM memory area holds compiled native machine code of JIT-compiled methods?

- A. Heap
- B. Metaspace
- C. Code Cache (part of native memory)
- D. Stack

Answer: C

Explanation: The JIT-compiled code is stored in the JVM’s code cache (native memory). The JVM allocates a special area for the optimized machine code.

37. What does “WORA” (Write Once, Run Anywhere) refer to in Java?

- A. The ability of bytecode to run on any machine with a compatible JVM.
- B. Java’s write-once memory model.
- C. A mechanism for atomic writes in Java I/O.
- D. The final keyword.

Answer: A

Explanation: WORA stands for “Write Once, Run Anywhere”, highlighting that Java bytecode can run on any platform with a compliant JVM. The source notes this: “Java applications are called WORA... developer can expect to run on any Java-enabled system... because of the JVM” ¹⁷.

38. How does the JVM treat the `int` literal `10` versus the `long` literal `10L` at compile time?

- A. Both are stored as `int` in class file; the `L` is ignored.
- B. `10` is a 32-bit int in bytecode; `10L` is a 64-bit long in bytecode.
- C. Both become `long` because of automatic widening.
- D. The compiler refuses to compile `10` without an `L`.

Answer: B

Explanation: In Java, an unsuffixed integer literal like `10` is treated as an `int`. A literal with an `L` (like `10L`) is treated as a `long`. In bytecode, `10` would be an `iconst` or push int, whereas `10L` would be `lconst`. The `L` suffix tells the compiler to use a long literal.

39. If you enable the JVM option `-verbose:gc`, what effect does it have?

- A. It runs the GC in a special verbose mode that never frees objects.
- B. It prints information about each garbage collection to standard output.
- C. It disables garbage collection.
- D. It logs only when a class is unloaded.

Answer: B

Explanation: The `-verbose:gc` (or newer `-Xlog:gc*`) option causes the JVM to log details about each garbage collection event (when it happens, how long it took, memory usage before/after, etc.) to standard output.

40. What is the usual representation of a Java stack trace element when an exception is thrown?

- A. Native memory address
- B. File name and line number of the code in the class that was executing
- C. The class loader name
- D. The CPU register state

Answer: B

Explanation: A stack trace element typically shows the class name, method name, filename, and line number where each stack frame is located. This allows developers to pinpoint where exceptions occurred.

41. How does HotSpot decide which methods to JIT-compile?

- A. It JITs all methods immediately at class load time.
- B. It never JITs, always interprets.
- C. It JITs methods that are “hot” (frequently executed) to optimize performance.
- D. The user must manually specify which methods to JIT using annotations.

Answer: C

Explanation: The HotSpot JVM uses a tiered compilation strategy. It identifies “hot” methods (those executed often, such as loops) and compiles them to native code for efficiency. Less-used code may remain interpreted initially. Users cannot specify JIT per method except via JVM flags or diagnostic tools.

Sources: Authoritative Java documentation and tutorials on data types, control flow, OOP, exception handling, collections, concurrency, Java 8 features, class loading, and memory management were used to compile these questions and explanations (e.g., GeeksforGeeks Java tutorials ¹⁷ ³³ ³⁴ ³⁵, Baeldung on ClassLoaders ²⁵, and Sanfoundry Java MCQs ² ⁵). Each fact in the explanations is supported by these sources.

1 Default value of 'boolean' and 'Boolean' in Java - Stack Overflow

<https://stackoverflow.com/questions/6226290/default-value-of-boolean-and-boolean-in-java>

2 3 4 5 8 9 10 11 12 13 14 15 1000 Java MCQ (Multiple Choice Questions) - Sanfoundry

<https://www.sanfoundry.com/java-questions-answers-freshers-experienced/>

6 Quiz about Java Data Types

<https://www.geeksforgeeks.org/quizzes/java-quiz/data-types-2-gg/>

7 Character (Java Platform SE 8)

<https://docs.oracle.com/javase/8/docs/api/java/lang/Character.html>

16 Why Java Strings are Immutable? - GeeksforGeeks

<https://www.geeksforgeeks.org/java/java-string-is-immutable-what-exactly-is-the-meaning/>

17 18 19 20 21 22 26 29 30 31 32 33 How JVM Works - JVM Architecture - GeeksforGeeks

<https://www.geeksforgeeks.org/java/how-jvm-works-jvm-architecture/>

23 24 The Class Loader Hierarchy (Sun Java System Application Server Platform Edition 9 Developer's Guide)

<https://docs.oracle.com/cd/E19501-01/819-3659/beatf/index.html>

25 Class Loaders in Java | Baeldung

<https://www.baeldung.com/java-classloaders>

27 28 34 Garbage Collection in Java - GeeksforGeeks

<https://www.geeksforgeeks.org/java/garbage-collection-in-java/>

35 Java Checked vs Unchecked Exceptions - GeeksforGeeks

<https://www.geeksforgeeks.org/java/java-checked-vs-unchecked-exceptions/>